Universidad del Valle de Guatemala

Facultad de Ingeniería

Teoría de la Computación

Ciudad de Guatemala, Guatemala

Octubre, 19 del 2026

Proyecto 2

Algoritmo CYK



Diego Quan (24336)

Alejandro Pérez (24136)

Introducción

El programa recibe una frase en inglés y decide si pertenece al lenguaje de una gramática libre de contexto (CFG). Para eso convierte la gramática a Forma Normal de Chomsky (CNF), aplica el algoritmo CYK y muestra SÍ o NO, el tiempo que tardó CYK y, si la respuesta es SÍ, el parse tree.

Diseño de la aplicación

La gramática se modela como G = (V, T, P, S) y se transforma paso a paso hasta CNF, que es lo que CYK necesita. El código está en Python, dividido en cinco módulos simples.

Flujo del programa:

Leer gramática → Quitar ε → Quitar unitarias → Quitar inútiles → Pasar a CNF → CYK + tiempo → SÍ / NO → Parse tree (si SÍ)

La gramática se lee y se lleva a CNF una sola vez. CYK, la respuesta SÍ/NO y el parse tree se repiten por cada frase ingresada.

Modelado

- **V**: lista de variables (no terminales). Un símbolo con producciones propias es variable.
- **T**: símbolos de los cuerpos que no tienen producciones propias (se calcula a partir de P).
- **P**: diccionario cabeza → lista de cuerpos. Cada cuerpo es una lista de símbolos. ε se guarda como ε; en el archivo se escribe e.
- **S**: la cabeza de la primera línea del archivo.

Módulos

Módulo	Responsabilidad
gramatica.py	Clase Gramatica y lectura del archivo, con validación de formato
simplificacion.py	Eliminar ε, unitarias, símbolos inútiles y convertir a CNF
cyk.py	Algoritmo CYK con tabla de programación dinámica
arbol.py	Construcción del parse tree y dibujo con Graphviz (opcional)
main.py	Flujo completo, lectura de frases y medición del tiempo

Algoritmos de simplificación (orden fijo)

1. **Eliminar ε**: se calcula el conjunto de anulables como punto fijo. Cada producción se reemplaza por todas sus variantes sin anulables (hasta 2^m combinaciones). Se pierde ε del lenguaje: L(G') = L(G) − {ε}.
2. **Eliminar unitarias**: se calculan las parejas (A, B) con A ⇒* B. La base es (A, A); si (A, B) y B → C es unitaria, entonces (A, C). Para cada pareja se copian las producciones no unitarias de B a A.
3. **Eliminar símbolos inútiles**: primero los que no generan cadenas de terminales y después los que no son alcanzables desde S. El orden importa.
4. **CNF**:
    - (a) En cuerpos de largo ≥ 2, cada terminal a se cambia por una variable T_a con T_a → a.
    - (b) Los cuerpos de largo ≥ 3 se parten en cadena con variables C1, C2, ..., compartiendo colas iguales.

El orden ε → unitarias → inútiles → CNF es necesario. Eliminar ε puede crear unitarias, y eliminar unitarias puede dejar símbolos inútiles. Al final el programa verifica que toda producción sea A → B C o A → a.

Algoritmo CYK

Sea w = w1 ... wn. La celda (l, i) guarda las variables que derivan las l palabras que empiezan en la posición i:

$T(l, i) = \{ A \mid A \Rightarrow^* w_i \ldots w_{(i+l-1)} \}$

- **Base (l = 1)**: A está en (1, i) si A → w[i] es producción.
- **Inducción (l ≥ 2)**: A está en (l, i) si existe A → B C y un corte k (1 ≤ k < l) con B en (k, i) y C en (l − k, i + k).
- **Aceptación**: la frase pertenece al lenguaje si y solo si S está en (n, 0).

Cada subcadena se resuelve una sola vez y se reutiliza (programación dinámica). Hay $O(n^2)$ celdas. Cada una prueba hasta n cortes y recorre las producciones, así que la complejidad es $O(n^3 \cdot |G|)$.

Parse tree

CYK se extiende para guardar cómo se obtuvo cada variable. En la base se guarda la palabra; en la inducción, la tripleta (k, B, C). construir_arbol recorre esa información de forma recursiva desde (S, n, 0): una hoja si l = 1, y dos hijos (B y C) si l ≥ 2. El árbol se imprime en texto y se dibuja con Graphviz como parse_tree.png.

Medición del tiempo

Se usa time.perf_counter() justo antes y después de llamar a cyk, con verbose=False para no cronometrar la impresión. La conversión a CNF no se incluye porque se hace una sola vez al inicio.

Discusión

El proyecto cumple lo pedido: la gramática se convierte a CNF y CYK responde SÍ/NO con tiempo y parse tree. Lo más delicado fue conservar la información que el árbol necesita y mantener equivalentes las gramáticas en cada paso.

Obstáculos encontrados

- **Variables sin producciones**: al eliminar ε, una variable cuya única producción era ε quedaba sin cuerpos y desaparecía del diccionario. Se resolvió con agregar_variable, que la mantiene registrada hasta que la limpieza de inútiles la elimina.
- **Variable o terminal**: el archivo no los distingue. Se adoptó la regla de que un símbolo con producciones propias es variable y el resto es terminal.
- **Nombres nuevos en CNF**: las variables auxiliares (T_a, C1, C2...) no debían chocar con las existentes. nombre_nuevo genera un nombre libre, y las colas iguales se comparten.
- **Parse tree**: guardar solo si una variable está en la celda no alcanza. Cada celda pasó a ser un diccionario variable → cómo se obtuvo.
- **Tiempo**: imprimir dentro del cronómetro distorsionaba la medida, así que solo se mide la llamada a cyk.

Decisiones y limitaciones

- No se elimina la recursividad izquierda: no hace falta para CNF ni para CYK.
- Si L(G) contiene ε, la gramática en CNF no la genera; la frase vacía se rechaza.
- Una frase ambigua tiene varios árboles, pero se guarda solo el primero que CYK encuentra.
- El árbol es el de la gramática en CNF. Por eso puede mostrar variables auxiliares (T_(, C1) en gramáticas que no estaban en CNF.
- Una variable usada pero nunca definida se lee como terminal.
- Una palabra fuera del vocabulario hace que la frase se rechace; el programa avisa cuál es.
- Las frases se pasan a minúsculas, se ignoran . , ; : ! ? y las palabras deben ir separadas por espacios.

Recomendaciones

- Mantener el orden ε → unitarias → inútiles → CNF.
- Validar CYK contra una enumeración de las cadenas que la gramática genera; así se detectaron cero discrepancias en las pruebas.
- Para gramáticas grandes, CYK se puede acelerar indexando las producciones por cuerpo en lugar de recorrerlas todas en cada celda.

Ejemplos y pruebas

La gramática del inglés ya está en CNF, y el programa responde bien tanto a frases del lenguaje como a frases fuera de él. Las pruebas automáticas no encontraron ninguna discrepancia.

Frases que pertenecen al lenguaje (SÍ)

Frase	**Resultado**
She eats a cake with a fork	SÍ
The cat drinks the beer	SÍ
he cuts the meat with a knife	SÍ
she cooks	SÍ
the dog eats the soup in the oven with a spoon	SÍ
a cat cooks a cake in the oven	SÍ
the cake drinks the dog	SÍ (sin sentido, pero sintácticamente correcta)

Frases que no pertenecen al lenguaje (NO)

Frase	**Resultado**	**Motivo**
cat the drinks beer	NO	Orden incorrecto
she eats a	NO	Falta el sustantivo
eats she cake	NO	Verbo antes del sujeto
he she eats	NO	Dos sujetos seguidos
the cat the dog drinks	NO	No hay estructura S → NP VP
the cat eats the dragon	NO	“dragon” no está en el vocabulario (el programa avisa)

Parse tree de “She eats a cake with a fork”

```
S
|-- NP
|   `-- she
`-- VP
    |-- VP
    |   |-- V
    |   |   `-- eats
    |   `-- NP
    |       |-- Det
    |       |   `-- a
    |       `-- N
    |           `-- cake
    `-- PP
        |-- P
        |   `-- with
        `-- NP
            |-- Det
            |   `-- a
            `-- N
                `-- fork
```

Una frase fuera del lenguaje no tiene derivación, por eso no se genera árbol.

Conversión a CNF de la gramática 1.txt

Original (1.txt)	**En CNF**
E → T X	E → T X | F Y | T_(C1 | id
X → + T X | e	X → T_+ C2 | T_+ T
T → F Y	T → F Y | T_(C1 | id
Y → * F Y | e	Y → T_* C3 | T_* F
F → (E) | id	F → T_(C1 | id
(sin producciones)	T_(→ (; T_) →) ; T_+ → + ; T_* → *
(sin producciones)	C1 → E T_) ; C2 → T X ; C3 → F Y

Verificaciones

- 1.txt: 60 cadenas (hasta 7 símbolos) dan el mismo resultado antes y después de la CNF.
- ingles.txt: 9312 cadenas (hasta 6 palabras) dan el mismo resultado antes y después.
- CYK contra la gramática en 1.txt: 3905 cadenas de largo 1 a 5, 0 discrepancias.
- CYK en ingles.txt: 3200 frases válidas de hasta 5 palabras, todas aceptadas; 4000 frases aleatorias, 0 discrepancias.
- Parse trees: 1500 frases válidas aleatorias por gramática. En todas la raíz es S, las hojas forman la frase y cada nodo interno es una producción real.

Tiempos (ingles.txt, promedio de 200 corridas)

Palabras	Tiempo CYK (ms)
4	0.03
7	0.15
10	0.37
16	1.45
28	9.8
52	61

Una frase 13 veces más larga (de 4 a 52 palabras) tarda unas 2000 veces más, cerca de $13^3 = 2197$, lo que concuerda con $O(n^3)$. Los valores dependen de la máquina; estos se midieron en una máquina de pruebas.